

同济大学

聚类及可视化实验报告

于沛可 2451951

数据科学与大数据技术

一、实验目的

本实验的主要目的是通过实际编程实现 K-Means 聚类算法，加深对无监督学习方法的理解，并将其应用于真实数据集进行分析。

具体目标包括：

1. 掌握 K-Means 聚类算法的基本原理和实现流程
2. 学会对真实数据进行必要的预处理，如缺失值处理、特征缩放等
3. 使用可视化方法直观展示聚类结果，并分析不同聚类数对结果的影响
4. 通过实验分析 K-Means 在真实数据集上的表现和局限性

二、实验内容

2.1 数据准备

本次实验选用 Kaggle 平台上的 Credit Card Dataset for Clustering 数据集。该数据集以信用卡用户的消费和还款行为作为特征，全部为数值型数据，适合采用 K-Means 算法进行聚类分析。

在该数据集中，每条记录对应一位信用卡持卡人，包含 17 个与消费和还款行为相关的数值型特征，具体说明如下：

特征名称	含义说明
BALANCE	当前账户余额，即用户尚未偿还、仍可用于消费的余额金额
BALANCE_FREQUENCY	账户余额更新频率，取值范围为 0－1，数值越大表示余额更新越频繁
PURCHASES	用户累计消费总金额
ONEOFF_PURCHASES	单次完成的最大消费金额（非分期消费）
INSTALLMENTS_PURCHASES	以分期方式完成的消费总金额
CASH_ADVANCE	用户从信用卡中预支的现金总金额
PURCHASES_FREQUENCY	消费行为发生频率，取值范围为 0－1，数值越大表示消费越频繁

特征名称	含义说明
ONEOFFPURCHASESFREQUENCY	单次（非分期）消费发生频率，取值范围为 0 - 1
PURCHASESINSTALLMENTSFREQUENCY	分期消费发生频率，取值范围为 0 - 1
CASHADVANCEFREQUENCY	现金预支行为发生频率
CASHADVANCETRX	现金预支交易次数
PURCHASES_TRX	消费交易次数
CREDIT_LIMIT	信用卡额度上限
PAYMENTS	用户已完成的支付总金额
MINIMUM_PAYMENTS	用户已支付的最低还款额总和
PRCFULLPAYMENT	用户全额还款比例，取值范围为 0 - 1
TENURE	用户持有并使用该信用卡的时间长度（月数）

2.2 聚类算法实现

1. 使用 K-Means 算法对数据进行聚类
2. 采用 K-Means++ 方法初始化聚类中心，以提升算法稳定性
3. 通过多次初始化（n_init）减少随机初始化带来的结果波动

2.3 模型评估与可视化分析

1. 计算不同聚类数下的簇内平方和（Inertia）
2. 计算轮廓系数评估聚类质量
3. 使用主成分分析 PCA 将高维数据降至二维，并对聚类结果进行可视化展示

三、实验过程

3.1 实验环境准备

本实验使用 Python 完成，主要依赖如下：

- pandas / numpy：数据处理与数值计算
- matplotlib：绘图

- sklearn：标准化（StandardScaler）、PCA、轮廓系数等评估工具
- kagglehub：自动下载 Kaggle 数据集

3.2 数据获取

我使用 kagglehub 直接拉取数据集到本地路径，下载代码如下：

```
import kagglehub

# Download latest version
path = kagglehub.dataset_download("arjunbhasin2013/ccdata")
print("Path to dataset files:", path)
```

下载后在该目录里可以找到 CC GENERAL.csv，后续读取时把路径改成对应位置即可。

3.3 数据预处理

原始数据中包含用户唯一标识字段 CUST_ID，该字段不反映用户行为特征，因此在实验中予以删除。

针对部分特征存在缺失值的问题，采用各特征的中位数进行填充，以减小异常值对聚类结果的影响。

由于 K-Means 算法基于欧氏距离，不同特征之间量纲差异会直接影响距离计算结果，因此在聚类前对数据进行标准化处理，使各特征具有相同尺度。

在此基础上，为缓解部分金额类特征的偏态分布问题，实验中尝试对特征取 log1p 变换，以降低极端值对模型的影响。

3.4 K-Means 聚类模型构建

3.4.1 聚类中心初始化

K-Means 算法对初始聚类中心较为敏感，不同的初始化方式可能导致不同的聚类结果。为减少随机初始化带来的不稳定性，本实验采用 K-Means++ 初始化方法。K-Means++ 通过按“样本到最近已选中心的距离平方”作为概率分布，依次选择新的中心点，使初始中心在数据空间中尽可能分散，从而提升算法收敛质量和结果稳定性。具体代码如下：

```python
# 2. K-Mean++ 核心模块
"""
使用 K-Means++ 初始化聚类中心
参数:
X   : (n_samples, n_features) 的数据矩阵
k   : 聚类数
rng : np.random.Generator 随机数生成器，用于保证可复现
返回:
centroids : (k, n_features) 初始化后的中心点
"""
def kmeans_plus_plus_init(X, k, rng):

    # 样本数、特征数
    n_samples = X.shape[0]
    n_features = X.shape[1]

    # 用来存放 k 个中心点
    centroids = np.zeros((k, n_features), dtype=float)

    # 1）随机选择第一个中心点
    first_index = rng.integers(0, n_samples)
    centroids[0] = X[first_index]

    # 2）计算每个点到“最近中心”的距离平方
    #    一开始只有一个中心，就是到 centroids[0] 的距离平方
    closest_dist_sq = np.zeros(n_samples, dtype=float)
    for i in range(n_samples):
        diff = X[i] - centroids[0]
        closest_dist_sq[i] = float(np.dot(diff, diff))
```

```
    # 3）依次选择剩余的中心点
    for c in range(1, k):
        # 距离平方的总和，用于归一化成概率
        total = float(np.sum(closest_dist_sq))

        # 如果 total 为 0，说明所有点和中心重合（极端情况），直接随机选一个点
        if total == 0.0:
            rand_index = rng.integers(0, n_samples)
            centroids[c] = X[rand_index]
            continue

        # 计算概率分布：距离平方越大，被选为新中心的概率越大
        probs = closest_dist_sq / total

        # 按照 probs 概率采样一个新中心点
        new_index = rng.choice(n_samples, p=probs)
        centroids[c] = X[new_index]

        # 4）更新每个点到最近中心的距离平方（取更小的那个）
        for i in range(n_samples):
            diff = X[i] - centroids[c]
            dist_sq = float(np.dot(diff, diff))
            if dist_sq < closest_dist_sq[i]:
                closest_dist_sq[i] = dist_sq

    return centroids
```

3.4.2 样本分配与中心更新

在初始化聚类中心后，K-Means 算法通过样本分配和中心更新两个步骤交替迭代：

1. 样本分配：

   根据欧氏距离，将每个样本分配给距离最近的聚类中心；

2. 中心更新：

   对每个簇，计算簇内所有样本的均值作为新的聚类中心。

上述过程不断重复，直至聚类中心的变化幅度小于设定阈值，算法收敛。

样本分配与中心更新代码如下：

```
"""
将每个样本分配给最近的聚类中心
返回:
labels : (n_samples,) 每个样本所属簇编号
"""
def assign_labels(X, centroids):
    n_samples = X.shape[0]
    k = centroids.shape[0]
    labels = np.zeros(n_samples, dtype=int)
    for i in range(n_samples):
        best_j = 0
        best_dist = None
        # 遍历所有中心，找距离最小的
        for j in range(k):
            diff = X[i] - centroids[j]
            dist_sq = float(np.dot(diff, diff))  # 用平方距离即可，不用开根号
            if (best_dist is None) or (dist_sq < best_dist):
                best_dist = dist_sq
                best_j = j
        labels[i] = best_j
    return labels
```

```
"""
根据当前 labels 更新聚类中心（每个簇取均值）
若出现空簇（某一簇没有任何样本），则随机选一个样本作为该簇中心，避免后续计算出错
"""
def update_centroids(X, labels, k, rng):
    n_features = X.shape[1]
    centroids = np.zeros((k, n_features), dtype=float)
    for j in range(k):
        # 找出属于第 j 类的样本下标
        idx_list = []
        for i in range(X.shape[0]):
            if labels[i] == j:
                idx_list.append(i)
        # 空簇处理
        if len(idx_list) == 0:
            rand_index = rng.integers(0, X.shape[0])
            centroids[j] = X[rand_index]
            continue
        # 计算均值作为新中心
        for f in range(n_features):
            s = 0.0
            for idx in idx_list:
                s += float(X[idx, f])
            centroids[j, f] = s / len(idx_list)
    return centroids
```

3.4.3 多次初始化与结果稳定性（n_init）

由于 K-Means 算法仍然存在一定随机性，即使采用 K-Means++ 初始化，不同运行结果之间仍可能存在差异。

为提高聚类结果的稳定性，对于同一聚类数 K，本实验采用 多次初始化（n_init）策略：即在相同 K 值下多次运行 K-Means 算法，并分别计算每次结果的簇内平方和（SSE），最终选取 SSE 最小 的聚类结果作为该 K 下的最终输出。

```python
# 计算簇内平方和（SSE / inertia）
def compute_inertia(X, labels, centroids):

    sse = 0.0
    for i in range(X.shape[0]):
        c = labels[i]
        diff = X[i] - centroids[c]
        sse += float(np.dot(diff, diff))
    return sse
```

```
"""
训练 K-Means，并使用 n_init 次初始化选择 SSE 最小的结果
返回：
best_labels, best_centroids, best_inertia
"""
def kmeans_fit(X, k, max_iter=300, tol=1e-4, n_init=10, random_state=42):
    rng = np.random.default_rng(random_state)
    best_labels = None
    best_centroids = None
    best_inertia = None
    # 多次初始化，选择最优
    for run in range(n_init):
        # 初始化中心
        centroids = kmeans_plus_plus_init(X, k, rng)
        # 迭代更新
        for it in range(max_iter):
            labels = assign_labels(X, centroids)
            new_centroids = update_centroids(X, labels, k, rng)
            # 判断中心是否收敛（移动距离很小）
            shift = float(np.linalg.norm(new_centroids - centroids))
            centroids = new_centroids
            if shift < tol:
                break
        inertia = compute_inertia(X, labels, centroids)
        # 记录最优结果
        if (best_inertia is None) or (inertia < best_inertia):
            best_inertia = inertia
            best_labels = labels.copy()
            best_centroids = centroids.copy()
    return best_labels, best_centroids, best_inertia
```

3.5 聚类结果可视化

3.5.1 Elbow Method（肘部法）

Elbow Method（肘部法）通过观察簇内平方和（SSE）随聚类数 K 增大的变化趋势来辅助选择合适的 K 值。

随着 K 的增加，SSE 会不断减小，但当 K 增大到某一数值后，SSE 的下降幅度会明显变缓，曲线呈现类似“手肘”的形状。该拐点通常被视为一个较为合理的聚类数选择。代码如下：

```python
def plot_elbow(ks, inertias, save_path=None):
    plt.figure()
    plt.plot(ks, inertias, marker="o")
    plt.xlabel("K (number of clusters)")
    plt.ylabel("Inertia / SSE")
    plt.title("Elbow Method (K vs SSE)")
    plt.grid(True, alpha=0.3)
    if save_path:
        plt.savefig(save_path, dpi=200, bbox_inches="tight")
    plt.show()
```

3.5.2 Silhouette Score（轮廓系数）

Silhouette Score（轮廓系数）用于综合衡量聚类结果的质量，其同时考虑了簇内紧密度（样本与同簇样本的平均距离）和簇间分离度（样本与最近其他簇的平均距离）轮廓系数取值范围为 [-1,1]，数值越大表示聚类效果越好。代码如下：

```python
def plot_silhouette(ks, sils, save_path=None):
    plt.figure()
    plt.plot(ks, sils, marker="o")
    plt.xlabel("K (number of clusters)")
    plt.ylabel("Silhouette Score")
    plt.title("Silhouette Score vs K")
    plt.grid(True, alpha=0.3)
    if save_path:
        plt.savefig(save_path, dpi=200, bbox_inches="tight")
    plt.show()
```

3.5.3 PCA 二维聚类可视化

由于原始数据包含 17 个特征，属于高维数据，难以直接进行直观可视化分析。因此，本实验采用 主成分分析（PCA）方法将数据降至二维空间，在尽可能保留原始数据信息的前提下，对聚类结果进行可视化展示。

在二维空间中，不同颜色表示不同的聚类结果，同时将聚类中心一并投影到二维平面上，以便观察各簇之间的分布情况及相对位置关系。代码如下：

```python
# PCA 降到2维后散点图展示聚类
def plot_pca_clusters(X, labels, centroids, save_path=None):

    pca = PCA(n_components=2, random_state=42)
    X2 = pca.fit_transform(X)
    C2 = pca.transform(centroids)

    plt.figure()
    plt.scatter(X2[:, 0], X2[:, 1], c=labels, s=10)
    plt.scatter(C2[:, 0], C2[:, 1], marker="X", s=180)
    plt.xlabel("PC1")
    plt.ylabel("PC2")
    plt.title("K-Means Clustering (PCA 2D Projection)")
    plt.grid(True, alpha=0.25)

    if save_path:
        plt.savefig(save_path, dpi=200, bbox_inches="tight")
    plt.show()
```

3.5.4 聚类中心画像

为进一步分析不同聚类结果在特征层面的差异，本实验对各聚类的中心点（Centroid）进行可视化，绘制聚类中心画像（Cluster Centroid Profiles）。由于在聚类前对数据进行了标准化处理，聚类中心在各特征维度上的取值表示相对于全体样本平均水平的偏离程度。其中，数值为 0 表示接近平均水平，正值表示该特征在该簇中整体高于平均水平，负值则表示低于平均水平。代码如下：

```python
# 聚类中心画像：每个簇在各特征维度上的中心值走势，用于结果解释
def plot_centroid_profile(feature_names, centroids, save_path=None):
    plt.figure(figsize=(10, 4))
    for i in range(centroids.shape[0]):
        plt.plot(range(len(feature_names)), centroids[i], marker="o", linewidth=1)
    plt.xticks(range(len(feature_names)), feature_names, rotation=75, ha="right")
    plt.xlabel("Features")
    plt.ylabel("Centroid value (standardized space)")
    plt.title("Cluster Centroid Profiles")
    plt.grid(True, alpha=0.25)
    if save_path:
        plt.savefig(save_path, dpi=200, bbox_inches="tight")
    plt.show()
```

四、测试样例说明

本实验使用的数据集为 Kaggle 平台上的 Credit Card Dataset for Clustering。测试过程中，以完整数据集作为输入，对不同聚类数 K（2－10）分别进行训练和评估。

在相同的数据预处理流程和随机种子设置下，比较不同 K 值对应的簇内平方和与轮廓系数，并结合可视化结果对聚类效果进行分析。

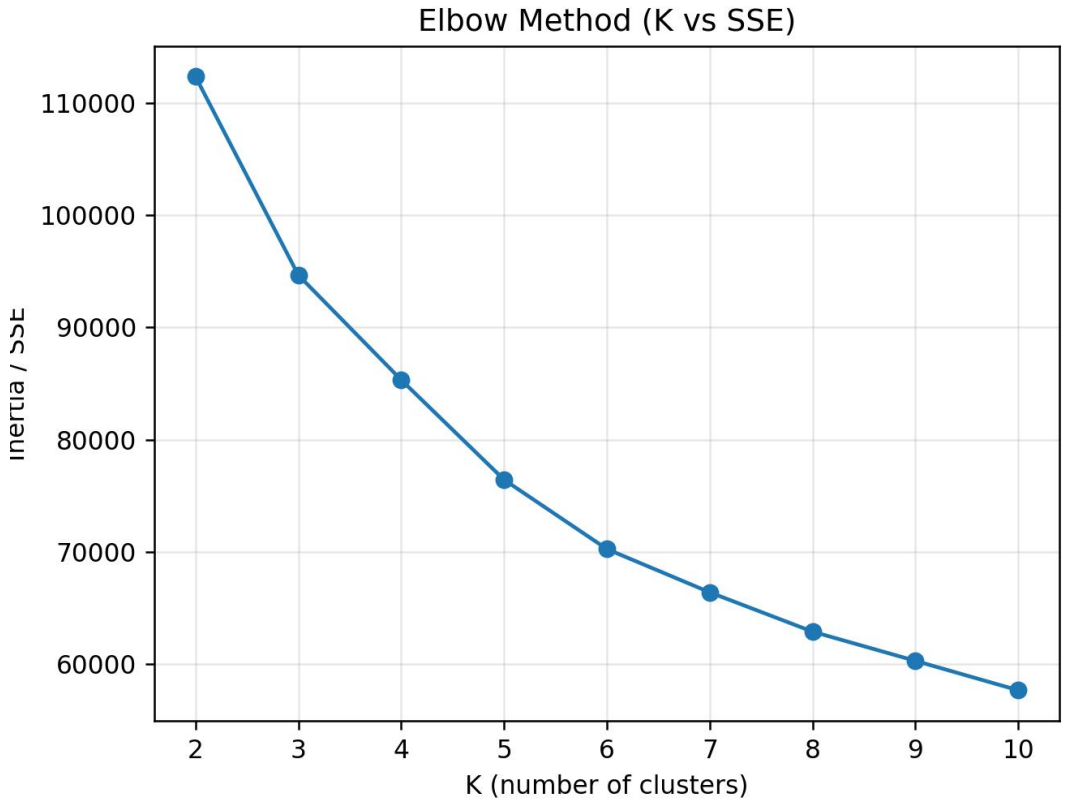
五、测试结果及分析

运行结果与可视化结果如下：

```
Data shape (rows, features): (8950, 17)
[K=2] inertia=112353.69, silhouette=0.2592
[K=3] inertia=94668.81, silhouette=0.2290
[K=4] inertia=85334.95, silhouette=0.2152
[K=5] inertia=76441.38, silhouette=0.2278
[K=6] inertia=70262.99, silhouette=0.2295
[K=7] inertia=66408.97, silhouette=0.2319
[K=8] inertia=62904.27, silhouette=0.2241
[K=9] inertia=60302.07, silhouette=0.1889
[K=10] inertia=57679.44, silhouette=0.2083
```

```
Recommended K by silhouette: K=2, silhouette=0.2592, inertia=112353.69
Cluster result saved to: outputs\cluster_result_k2.csv
```

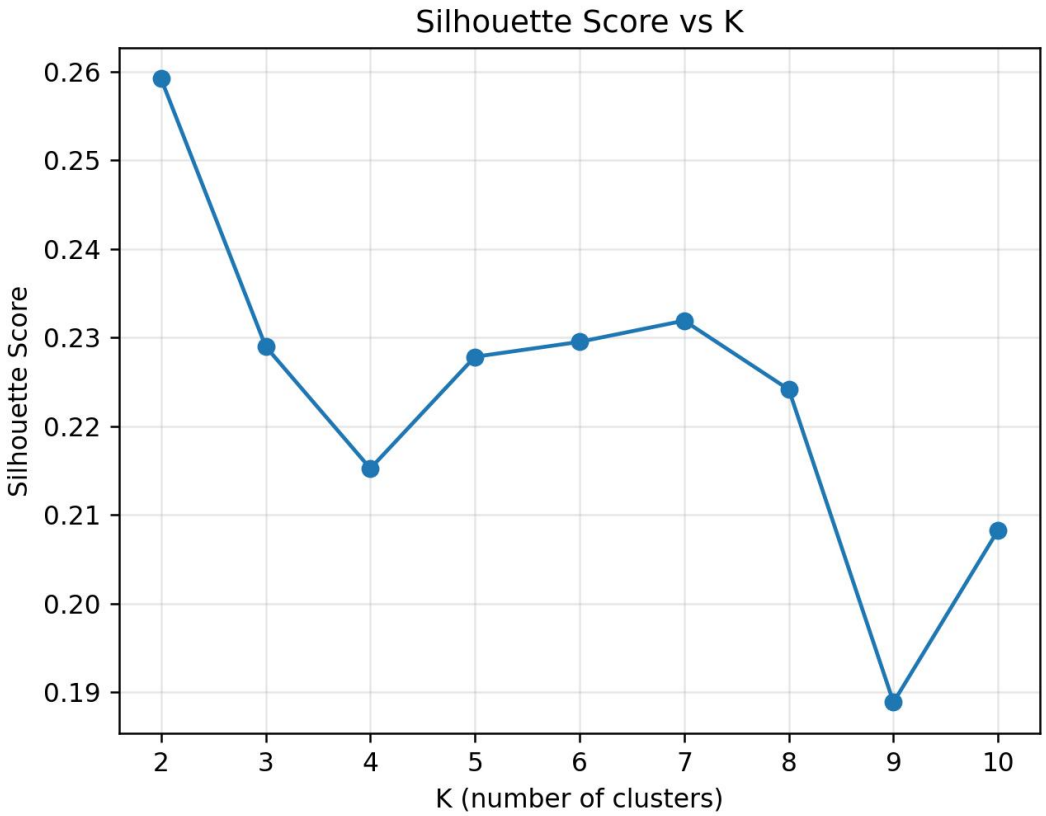
由运行结果数据可知，取 K=2 是最优解。进一步通过可视化验证：



由此可见，在本实验中，肘部法曲线随 K 增大呈现平滑下降趋势，未出现明显拐点，说明该数据集并不存在一个非常清晰的天然聚类数。这一现象与信用卡用户行为数据连续变化的特性相符。

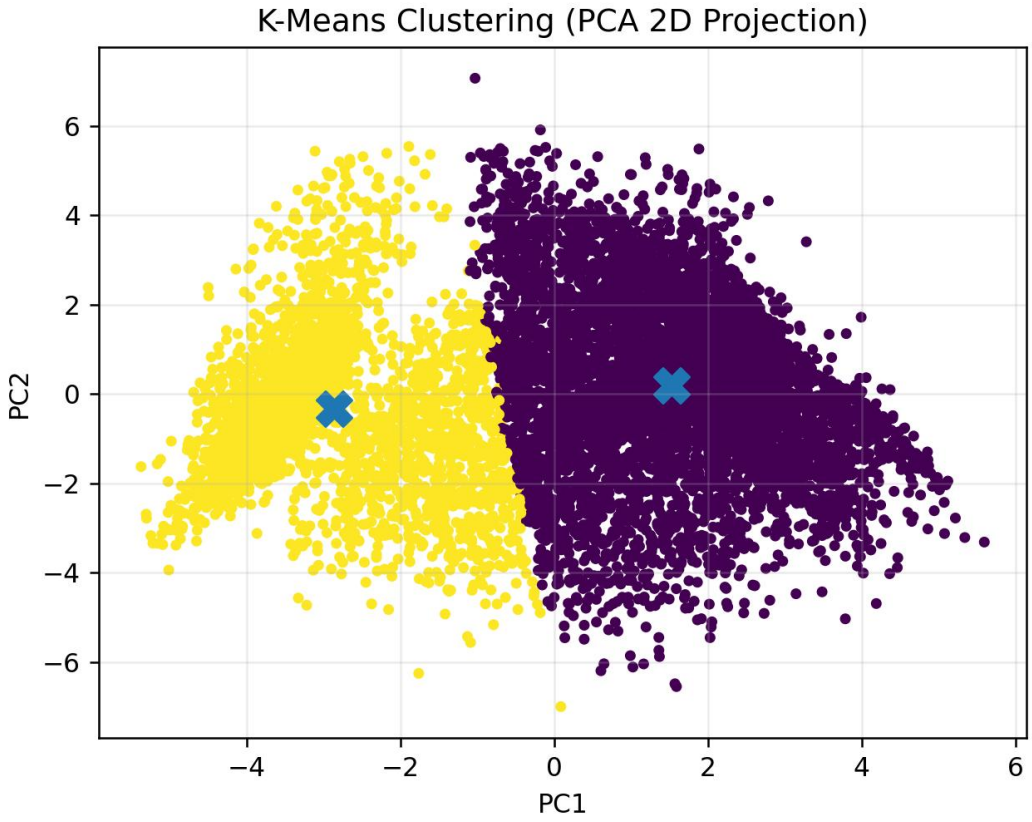
因此，本实验进一步结合轮廓系数作为主要参考指标，并最终选取轮廓系数较高的 K 值作为聚类数。

在确定 K 后，采用 PCA 方法对聚类结果进行二维可视化，用于辅助分析聚类结构的分离程度与合理性。



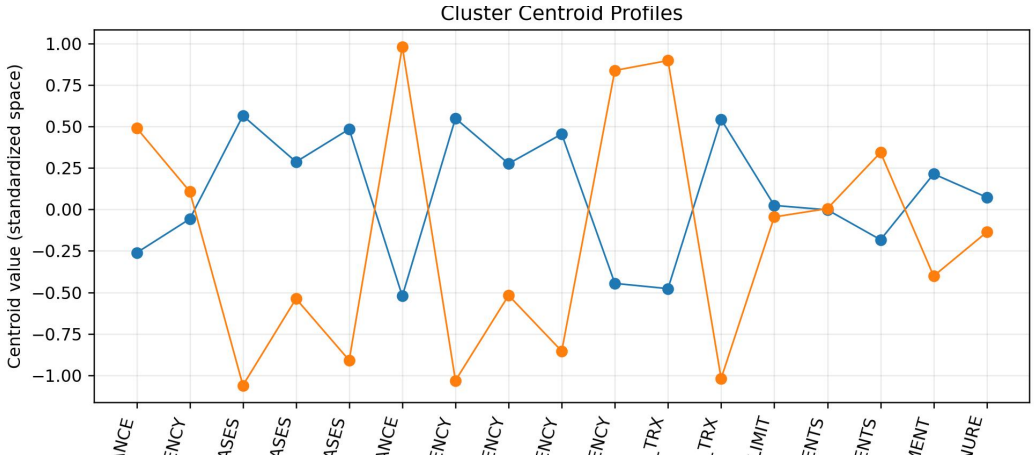
实验结果表明，当 K=2 时轮廓系数达到最大值（约 0.26），说明在该 K 值下，簇内样本相似度与簇间差异度的综合表现最佳。

随着 K 的进一步增大，轮廓系数整体呈下降趋势，表明过度划分会削弱聚类结构的清晰度。



从 PCA 二维投影图可以观察到，在 K=2 的情况下，样本大致形成两类分布，且聚类中心位于各簇的高密度区域附近。

虽然部分样本在低维空间中存在重叠，但整体上仍能观察到不同簇之间的分布差异，说明聚类结果具有一定的区分度。



同时，从聚类中心画像可以观察到，不同簇在多项特征维度上呈现出系统性差异，尤其在消费金额、交易次数、消费频率以及现金预支和还款相关特征上差异较为

明显。这表明 K-Means 聚类不仅在数值指标上具有合理性，也能够在行为特征层面对用户进行区分。

总体而言，一类用户在多数消费与交易相关特征上整体水平较高，表现出较为活跃的信用卡使用行为；另一类用户在上述特征上的整体水平较低，说明其信用卡使用相对保守。该结果进一步验证了聚类结果在实际意义上的可解释性。

六、模型优化改进

在完成基础 K-Means 聚类算法实现与可视化分析的基础上，为进一步提升聚类结果的稳定性、可解释性和整体质量，本实验从数据预处理、聚类流程以及结果评估与可视化三个层面对原有方法进行了改进与优化，并对改进前后的聚类效果进行了对比分析。

6.1 数据预处理层面的优化：引入 RobustScaler

K-Means 算法基于欧氏距离进行样本相似度计算，因此对特征尺度和异常值较为敏感。而在信用卡用户数据中，部分金额类和交易次数类特征（如 CASH_ADVANCE、PURCHASES、PAYMENTS 等）存在明显的极端值，如果直接采用基于均值和标准差的 StandardScaler，容易导致异常样本在距离计算中占据主导地位，从而影响聚类结果的稳定性。

为降低异常值对聚类的影响，本实验将特征缩放方法由 StandardScaler 替换为 **RobustScaler**。RobustScaler 使用中位数和四分位距（IQR）对数据进行缩放，相比均值和标准差，对异常值更不敏感，更适合本实验所使用的信用卡行为数据。

代码说明：在数据预处理函数 load_and_preprocess() 中，引入 use_robust 参数，根据实验设置选择缩放方式：

```
#  改进点 1: RobustScaler 抗异常值
# RobustScaler 使用中位数与四分位距缩放，异常值不会像 StandardScaler 那样强烈影响尺度
if use_robust:
    scaler = RobustScaler()
else:
    scaler = StandardScaler()

X = scaler.fit_transform(df.values)
return df, X
```

采用 RobustScaler 后，样本间的距离计算更加稳定，有效避免了少量极端消费

用户对整体聚类结构的干扰，为后续聚类质量的提升奠定了基础。

6.2 聚类流程层面的优化：PCA 降维后再进行 K-Means 聚类

原始数据包含 17 个数值型特征，这些特征之间可能存在较强相关性和冗余信息。在高维空间中，噪声维度和相关特征会对欧氏距离产生干扰，从而影响 K-Means 的聚类效果。

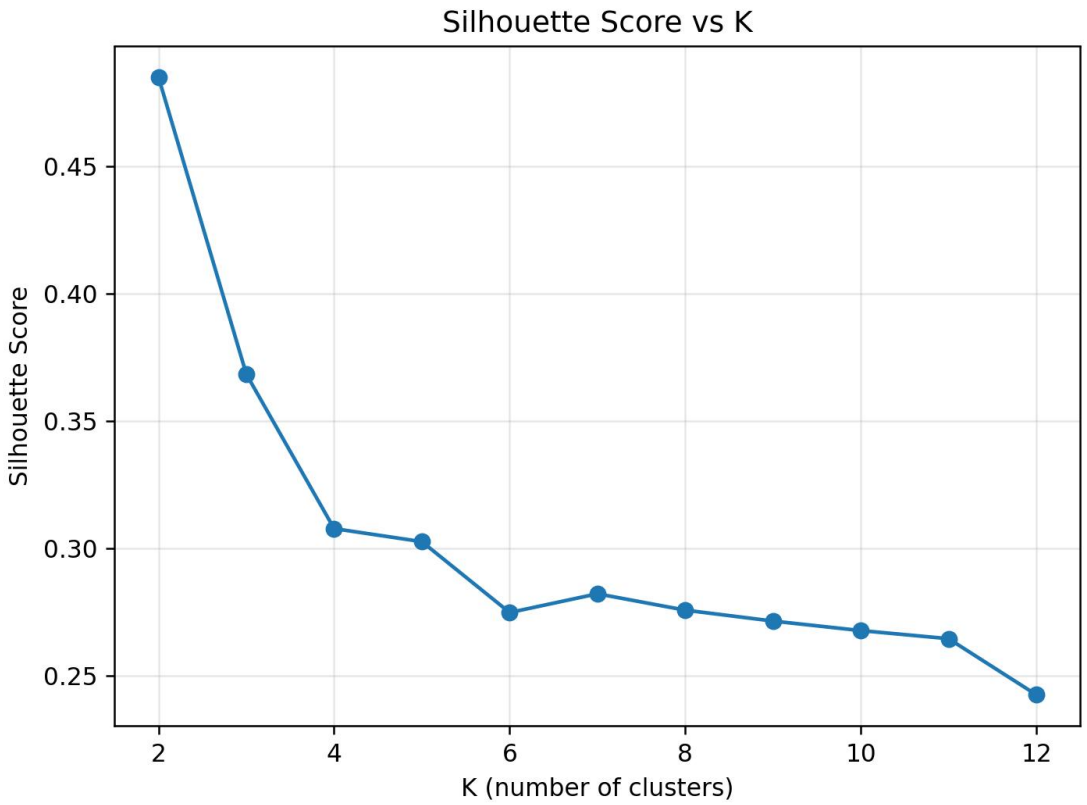
为减少冗余特征和噪声维度的影响，本实验引入 主成分分析（PCA） 对数据进行降维处理，在保留 95% 累计方差 的前提下，将特征维度由 17 维降至 7 维，并在降维后的主成分空间中执行 K-Means 聚类（PCA-KMeans）。

该方法能够在尽量保留原始数据信息的同时，提高距离度量的有效性和聚类稳定性。

代码如下：

```
# 3. 改进点 2：PCA-KMeans（先降维再聚类）
def pca_preprocess(X, var_ratio=0.95, random_state=42):
    """
    PCA 降维：保留 var_ratio 的累计方差（如 0.95）
    用于“先 PCA 再 K-Means”的流程优化
    """
    pca = PCA(n_components=var_ratio, random_state=random_state)
    X_pca = pca.fit_transform(X)
    return X_pca, pca
```

实验结果：



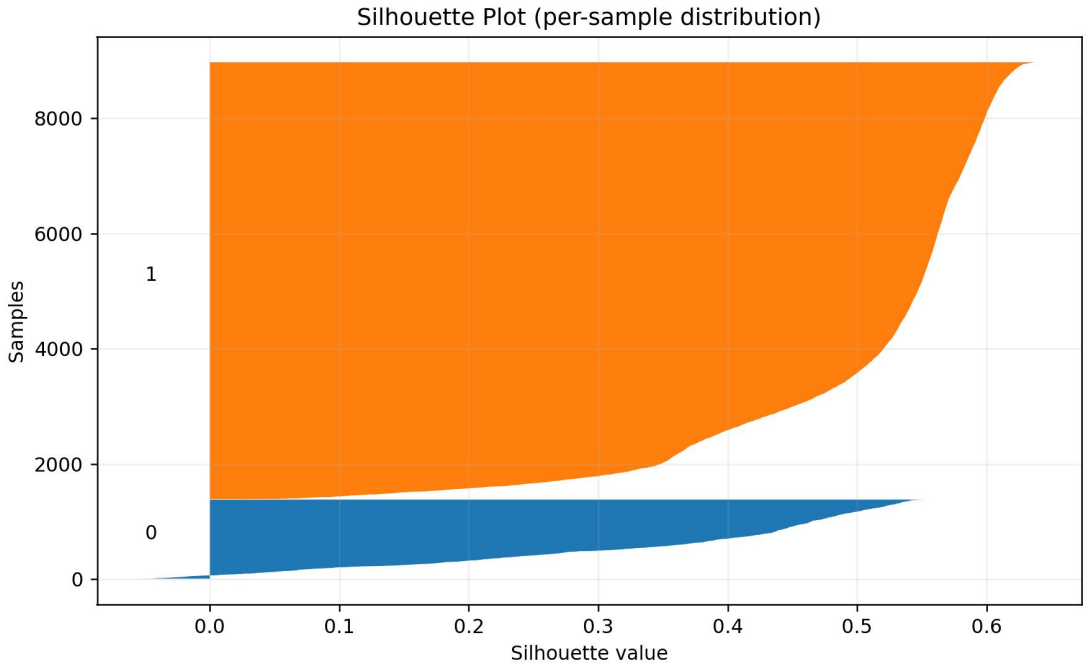
实验结果表明，引入 PCA 后，聚类的轮廓系数显著提升。在本实验中，K=2 时的轮廓系数由改进前约 0.26 提升至约 0.48，说明 PCA 有效削弱了噪声维度对距离计算的干扰，使聚类结构更加清晰。

6.3 结果评估与可视化层面的优化：引入轮廓系数分布图

传统的聚类评估通常只关注平均轮廓系数等单一数值指标，但该指标无法反映不同簇内部样本的分布情况，也难以判断是否存在局部聚类不合理的问题。

因此，在原有“轮廓系数随 K 变化曲线”的基础上，本实验进一步引入 轮廓系数分布图（Silhouette Plot），对每个样本的轮廓系数进行可视化展示。

该方法能够从样本层面观察各簇的内部一致性以及不同簇之间的分离程度。

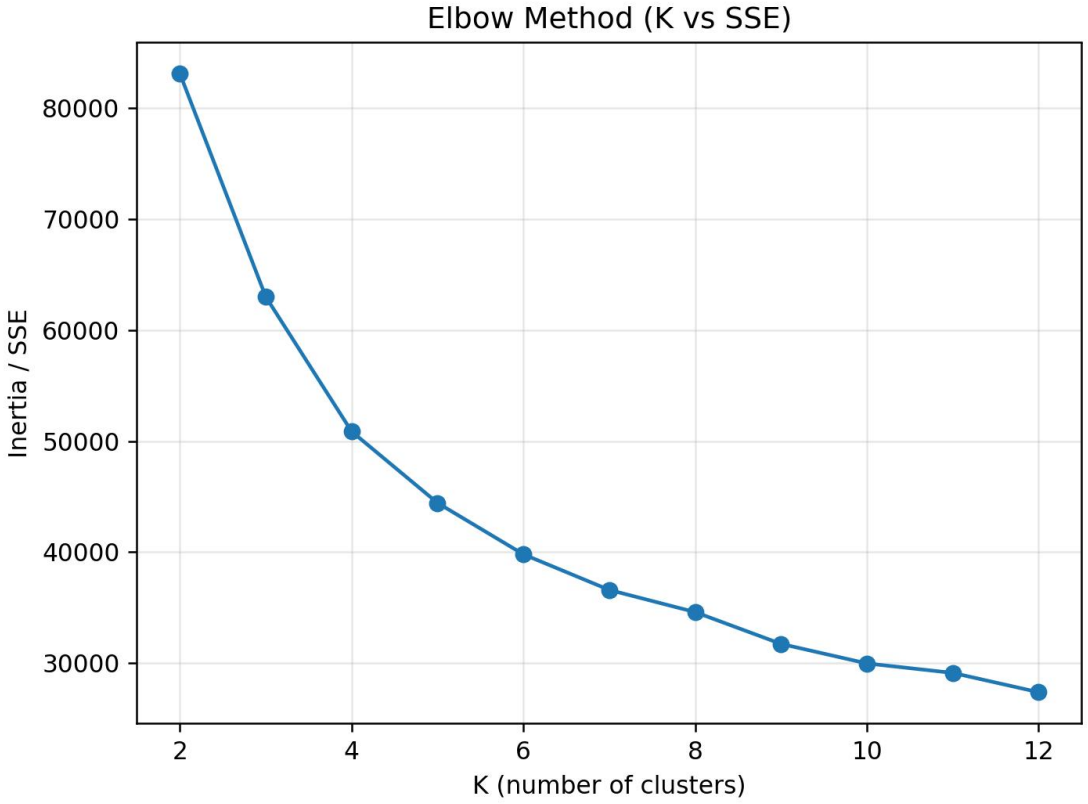


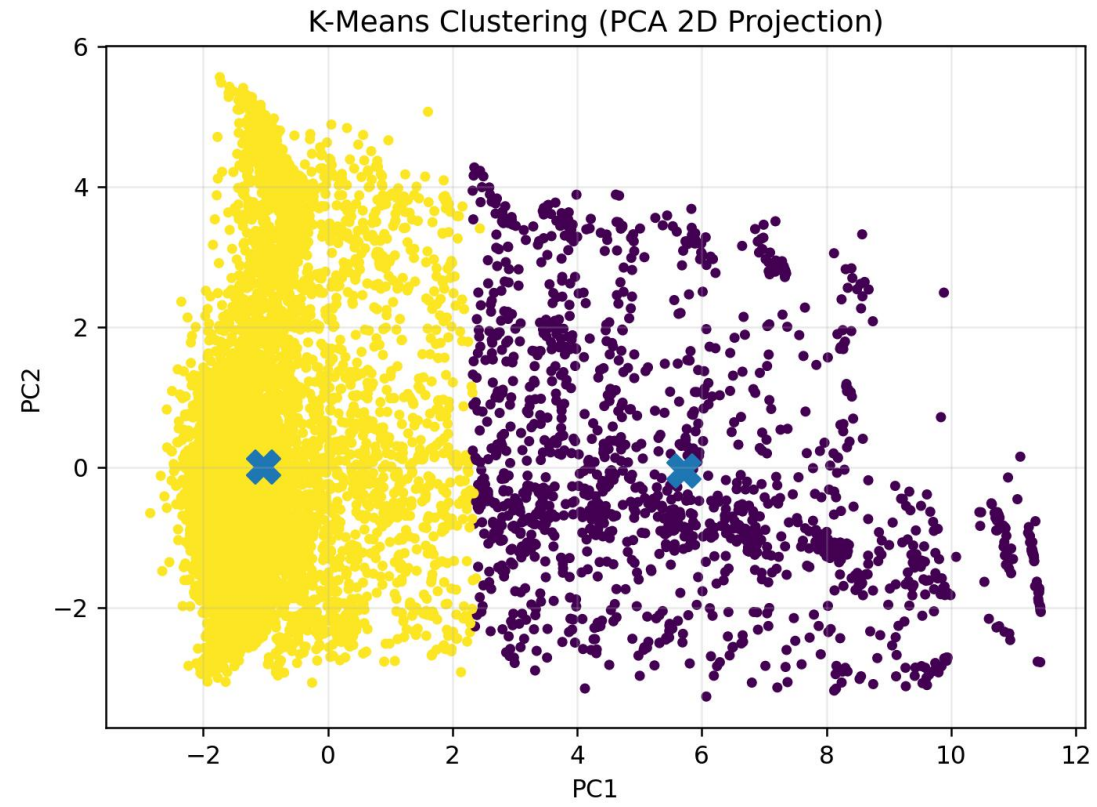
从轮廓系数分布图可以观察到，在 K=2 的情况下，两类样本的轮廓系数大多为正值，且集中分布在较高区间，几乎不存在明显的负值样本。这表明改进后的聚类结果不仅在平均指标上表现良好，在样本层面同样具有较高的一致性和稳定性。

改进后实验结果如下：

```
Recommended K by silhouette: K=2, silhouette=0.4848, inertia=83130.55
Cluster result saved to: outputs\cluster_result_k2.csv
```

```
Data shape (rows, features): (8950, 17)
PCA for K-Means: 17 -> 7 dims (var_ratio=0.95)
[K=2] inertia=83130.55, silhouette=0.4848
[K=3] inertia=63012.66, silhouette=0.3683
[K=4] inertia=50877.64, silhouette=0.3077
[K=5] inertia=44439.11, silhouette=0.3026
[K=6] inertia=39809.06, silhouette=0.2748
[K=7] inertia=36598.85, silhouette=0.2822
[K=8] inertia=34600.78, silhouette=0.2757
[K=9] inertia=31736.76, silhouette=0.2714
[K=10] inertia=29974.89, silhouette=0.2677
[K=11] inertia=29121.94, silhouette=0.2645
[K=12] inertia=27380.71, silhouette=0.2425
```







综合以上改进措施，可以得到如下结论：

1. 引入 RobustScaler 有效提升了模型对异常值的鲁棒性，使距离计算更加合理；
2. 采用 PCA 降维后进行 K-Means 聚类，显著提高了聚类质量，使轮廓系数从约 0.26 提升至约 0.48；
3. 通过轮廓系数分布图对聚类结果进行细粒度分析，进一步验证了聚类结构的稳定性和可解释性。

总体而言，经过多层次优化后的 K-Means 模型在该信用卡用户数据集上表现出明显优于基础实现的聚类效果，验证了所提出改进方法的有效性。

七、思考与总结

本实验围绕 K-Means 聚类算法展开，结合信用卡用户真实行为数据，从算法实现、参数选择、结果评估以及模型优化等多个方面进行了系统性实验与分析。在基础实验阶段，完成了 K-Means 算法的核心实现，包括 K-Means++ 初始化、样本分配与聚类中心更新、多次初始化（n_init）选择最优结果等关键步骤，并通过肘部法和轮廓系数对不同聚类数下的结果进行了评估。同时，借助 PCA 二维投影和聚类中心画像等可视化方法，对聚类结果进行了直观分析。

在此基础上，针对信用卡数据存在异常值、高维特征相关性强等特点，对原始方法进行了多方面优化。实验结果表明，引入 RobustScaler 后，模型对极端消费行为的鲁棒性显著提升；通过 PCA 在保留 95% 累计方差的前提下降维后再进行聚类，有效减少了噪声特征对欧氏距离计算的干扰，使轮廓系数从约 0.26 提升至约 0.48，聚类结构更加清晰稳定。进一步引入轮廓系数分布图，从样本层面对聚类结果进行分析，验证了各簇内部的一致性和整体划分的合理性。

需要指出的是，在本实验中，肘部法未出现非常明显的拐点。这一现象并不意味着聚类失败，而是反映了信用卡用户行为数据本身呈连续分布、缺乏天然清晰分组的特性。在此情况下，单纯依赖 SSE 并不足以确定最优聚类数，因此本实验将轮廓系数作为主要参考指标，并最终选取 K=2 作为聚类数。

从模型局限性来看，K-Means 假设簇呈现近似球状分布，且对初始中心和距离度量较为敏感。在面对密度不均或非凸形状的数据时，其效果可能受限。未来可尝试基于概率模型的高斯混合模型（GMM）或基于密度的聚类方法（如 DBSCAN）进行对比研究，以进一步挖掘用户行为模式。

总体而言，本实验通过对 K-Means 算法的实现与改进，验证了其在信用卡用户行为分析中的可行性，同时也加深了对无监督学习方法优势与局限性的理解。